Analysis of Algorithms Homework 3 Problem 3
Nicholas Cabrera
Ethan Patterson

1. **Recursive Solution**
a.) The algorithm starts by choosing the starting value, testing every value in the array for this. For the main algorithm after this, we check the current best subsequence length(1 by default; itself), then for any potential values in the array before this value that fit the criteria to be a subsequence value, we find the subsequences for those by recurring. If they produce a better subsequence than 1, we use that value as our “best”.

b.) Pseudocode

```
Main:
        Longest = 1
        For value from 0 to inputArray.size
                Longest = max(longest, increasingSubseq(inputarray, value))
        Return longest
increasingSubseq(inputArray, index):
        If index is 0 return 1
        Best = 1
        For i = 0; i<index;i++
                If inputarray[i] < inputarray[index]
                        best = max(best, increasingSubseq(inputarray, i)+1)
        Return best
```

c.) We test every value in the input array as the subsequence “root”. Then we check every possible value that could be a subsequence, check every possible value that could be a subsequence of that value, and so on until we test every value, and sum them. The largest is kept. It’ll work but the time complexity is not happy.

d.) 2^n time complexity

e.) We re-calculate the majority of the subproblems for every input value, and multiple times inside of those. Functionally it breaks down to n, n-1, n-2, …, 1, 0 which goes to 2^n when simplified.

2. **Dynamic Programming Solution**
a.) The dynamic programming solution approaches this slightly differently. First, it will iterate through an element j, and look through all elements before it such that they are less than j. It will total these and add it to an array S. It will do this for all elements. Thus, the answer will be the highest number stored in S, as each element in S represents the longest subsequence up to and including that element.

b.) Pseudocode:

```
array is input with size n

For j=1 to n:
        S[j] = 1
        For k to j:
                If (array[k] < array[j] and S[j] < S[k] + 1:
                        S[j] = S[k] + 1
max_S = 0
For each val in S:
        max_S = max(max_S, val)

output max_S
```

c.) For each element in the array, we are calculating the longest subsequence. If we know this is our “base” case, and we propagate that throughout the array, then each element in the saved “S” array must be the longest subsequence that includes that element at the end. Thus, the max is the longest subsequence of the array.

d.) O(n^2)

e.) This is n^2 time as at the worst case we run through all elements (size n), n times.

3. **Table of running times**

Input Size (n)	Recursive Running Time	Dynamic Prog Running Time
10	1.38ms	1.73ms
20	1.16ms	1.47ms
30	1.76ms	1.43ms
40	3.99ms	1.48ms
50	12.84ms	1.18ms
100	922ms	1.65ms
250	DNF	2.94ms
500	DNF	3.96ms
1000	DNF	13.1ms
10000	DNF	1190ms

Note- X axis is on a logarithmic scale.

Dynamic programming is SIGNIFICANTLY better than the recursive, providing much faster times out of the gate for smaller input sizes, and actually completing larger input sizes, which the recursive version was unable to do due to how much time they took. Recursive is an order of magnitude worse in its inputs.